

Laboratory 6: Linear Models II

Ballotta Sofia, Bollengier Alexis, Di Ciano Paolo, Greco Giorgio

EE-512 Applied Biomedical Signal Processing

EPFL, Lausanne Switzerland 22/10/2025

Instructions

- This notebook provides all the questions of the practical session and the space to answer them. We recommend working directly here and then exporting the document as your report.
- Include any code used when addressing the questions, any figures you find useful, together with your answers.
- Please submit your report as a single PDF file, named according to:
name1_name2_name3_LabLinearModelsI.pdf .
- We recommend working in a group of 4–5 students (max 6); the report does not have to be submitted by every group member -- one submission per group is sufficient.

```
In [30]: %matplotlib inline

# Imports
import os
import numpy as np
import matplotlib.pyplot as plt

from scipy.signal import resample, welch, correlate
from statsmodels.regression.linear_model import yule_walker
from statsmodels.tsa.ar_model import AutoReg

# File paths
fhr1 = os.path.join(os.getcwd(), 'data', 'heart_1.dat')
fhr2 = os.path.join(os.getcwd(), 'data', 'heart_2.dat')
fem1 = os.path.join(os.getcwd(), 'data', 'emg1.dat')
fem2 = os.path.join(os.getcwd(), 'data', 'emg2.dat')
fppg = os.path.join(os.getcwd(), 'data', 'ppg.dat')
feeg = os.path.join(os.getcwd(), 'data', 'eeg_stim.dat')
```

Experiment 1: parametric spectral estimation of cardiorespiratory signals

The file `/data/heart_1.dat` contains three physiological signals: RR-interval, arterial pressure and respiration, in this order (sampled at 4 Hz). The recordings were made in a patient at rest. In the spectral domain, this RR-interval signal is expected to show a peak around 0.08 Hz, corresponding to the *baroreflex* (regulation between arterial blood pressure and heart rate), and a peak at 0.225 Hz, corresponding to the *breathing rate*

(heart rhythm modulation by respiration, the so-called “*sinus arrhythmia*”) – see `/data/readme_heart.txt` for more details.

We start by importing the signals, subsampling to 1 Hz, and removing their averages:

```
In [3]: with open(fhr1, 'r') as f:
        txt = f.readlines()
        txt = [s[:-1].split() for s in txt]

        rr1 = np.array([float(s[0]) for s in txt])
        bp1 = np.array([float(s[1]) for s in txt])
        rs1 = np.array([float(s[2]) for s in txt])

        df = 4
        rr1 = resample(rr1, int(len(rr1)/df))
        bp1 = resample(bp1, int(len(bp1)/df))
        rs1 = resample(rs1, int(len(rs1)/df))

        for y in [rr1, bp1, rs1]:
            y -= np.mean(y)
```

AR modeling provides tools for extracting various useful estimates from a signal, in a so-called *parametric* way, because the estimates are based on the model parameters, rather than on the raw signal itself. The function defined below implements the estimation of the signal spectral density based on its AR parameters.

```
In [4]: def ar_psd(arg1, arg2, arg3, n=1000):
        """
        Parametric spectral density estimation based on AR modeling

        Inputs:
            arg1: ?
            arg2: ?
            arg3: ?
            n: defines the spectral sampling

        Outputs:
            arg4: ?
            arg5: ?
        """

        arg5 = np.arange(n) * arg3/2/n
        aar, sg = yule_walker(arg1, order=arg2, method="mle")

        a = complex(1)
        for k in range(1, arg2+1):
            a += -aar[k-1] * np.exp(-2*np.pi*k*1j*arg5/arg3)

        arg4 = sg**2 / (arg3 * np.abs(a)**2)

        return arg4, arg5
```

Question 1.1. The implementation above has an incomplete docstring (help text), and the input and output arguments have generic names (`arg1-5`). Copy the function to the answer space below, complete the help, and replace the argument names by more intuitive choices, based on what you've learned in the theory.

```
In [5]: ***Answer 1.1.**
def ar_psd(signal, order, fs, n=1000):
    """
    Estimate the parametric power spectral density (PSD) of a signal
    using an autoregressive (AR) model of given order.

    Inputs
    -----
    signal : 1D numpy array
        Input time series (should be zero-mean).
    order : int
        Order of the AR model.
    fs : float
        Sampling frequency in Hz.
    n : int, optional
        Number of frequency samples for PSD estimation (default = 1000).

    Outputs
    -----
    psd : 1D numpy array
        Estimated power spectral density of the signal.
    f : 1D numpy array
        Frequency axis in Hz corresponding to the PSD values.
    """

    # Frequency axis
    freq = np.arange(n) * fs / 2 / n

    # Estimate AR coefficients (a) and noise variance (sigma)
    a, sigma = yule_walker(signal, order=order, method="mle")

    # Compute transfer function denominator  $|1 + \sum a_k e^{-j2\pi k f / fs}|^2$ 
    H = np.ones_like(freq, dtype=complex)
    for k in range(1, order + 1):
        H += -a[k-1] * np.exp(-2j * np.pi * k * freq / fs)

    # PSD formula for AR model
    psd = sigma**2 / (fs * np.abs(H)**2)

    return psd, freq
```

We now use this tool to obtain the spectral density of the three signals. Below, we explore the results obtained with different AR model orders: 5, 15, 40.

```
In [6]: y = [rr1, bp1, rs1]
        ttl = ['RR-interval', 'Arterial pressure', 'Respiration']

        for p in (5, 15, 40):

            plt.figure(constrained_layout=True)
```

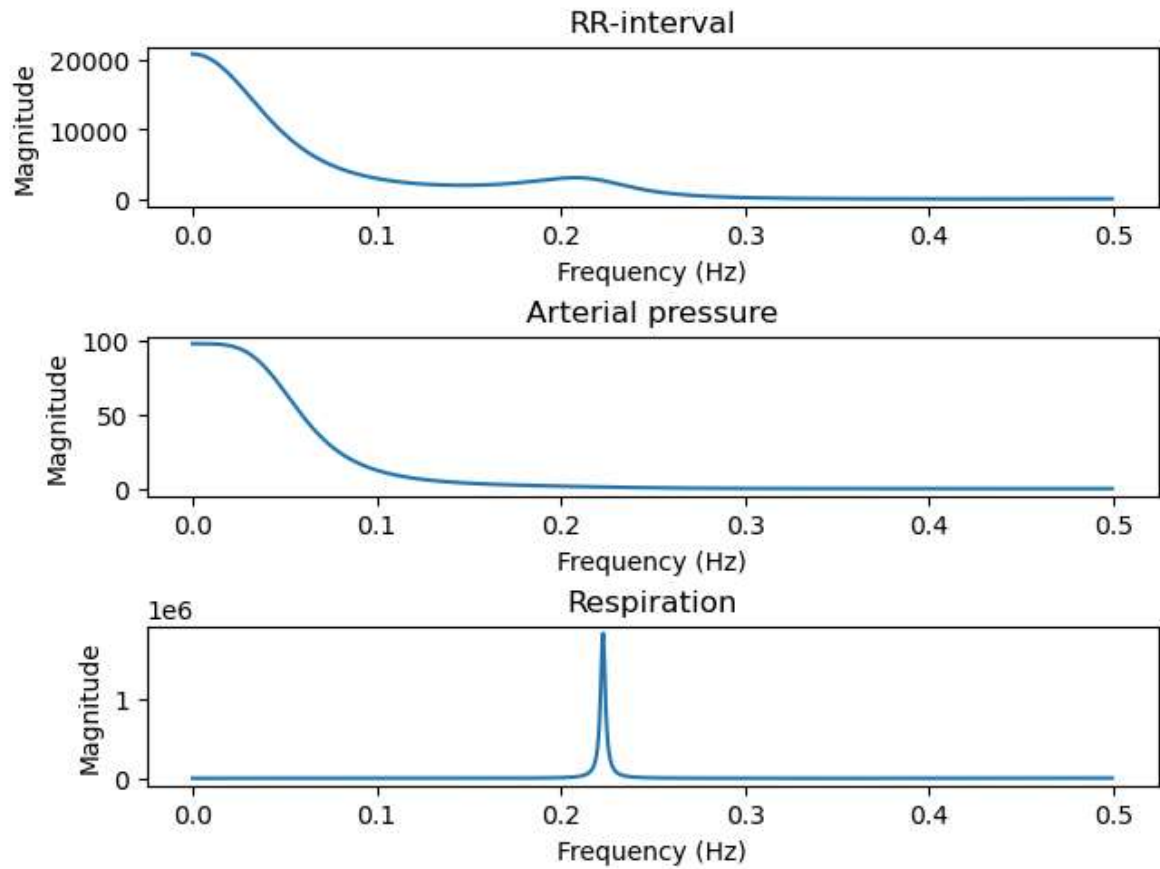
```

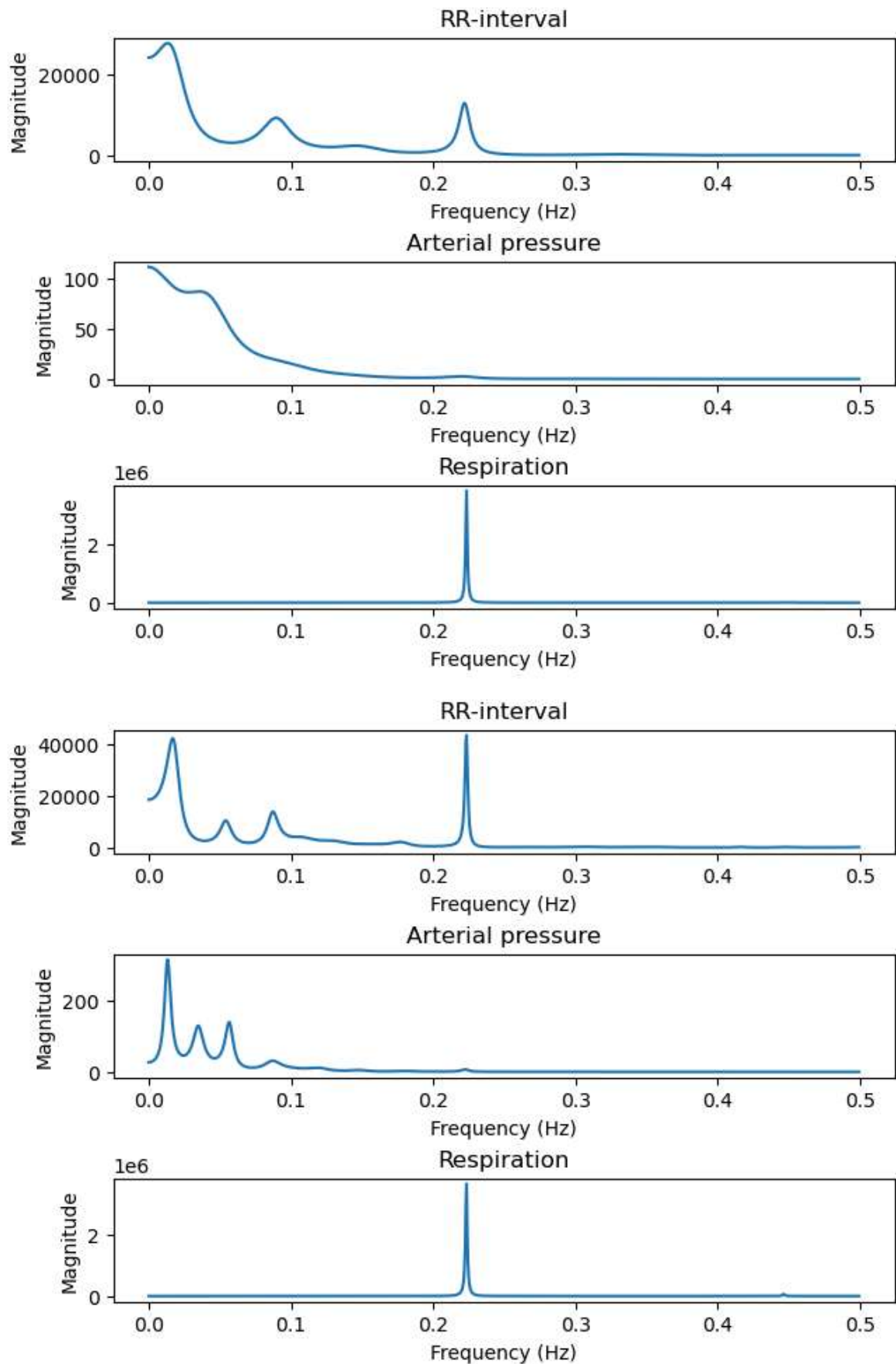
for k in range(3):

    py, f = ar_psd(y[k], p, 1.0)

    plt.subplot(3,1,k+1)
    plt.plot(f, py)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.title(ttl[k])

```





Question 1.2. How does the order affect the spectra? Given the physiological description of the expected peaks for the RR-interval, what is the most suitable order choice?

Answer 1.2. As the AR model order increases, the smoothness and resolution of the estimated spectrum change. With a low order, such as $p = 5$, the spectrum is too smooth

and fails to separate the main physiological components. The expected peaks around 0.08 Hz (related to the baroreflex, the interaction between arterial pressure and heart rate) and 0.225 Hz (the respiration frequency) are not clearly visible. When the order is increased to $p = 15$, the spectrum becomes more detailed and both expected peaks appear clearly at the correct frequencies. This behavior matches the physiological description of the signals and provides a realistic representation of the underlying processes. With a very high order, such as $p = 40$, the model becomes too complex and starts fitting the noise in the data. This results in an irregular spectrum with extra artificial peaks (numerical artifacts). In conclusion, an AR model of order 15 offers the best compromise: it correctly highlights the two main physiological peaks without introducing distortions or overfitting.

We can now perform a *non-parametric* spectral estimation of the same signals, using for instance Welch's method:

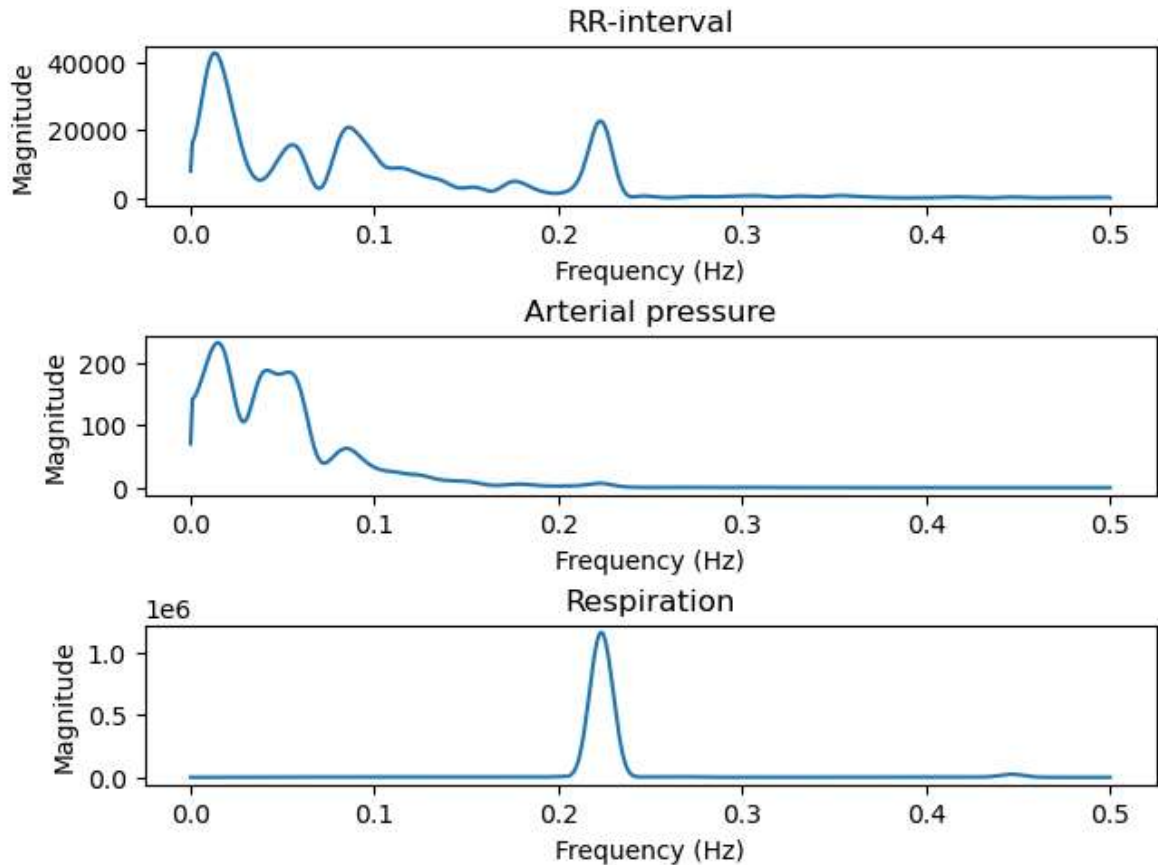
```
In [7]: y = [rr1, bp1, rs1]
        ttl = ['RR-interval', 'Arterial pressure', 'Respiration']

        plt.figure(constrained_layout=True)

        for k in range(3):

            f, py = welch(y[k], fs=1.0, nperseg=100, scaling='density', nfft=1000)

            plt.subplot(3,1,k+1)
            plt.plot(f, py)
            plt.xlabel('Frequency (Hz)')
            plt.ylabel('Magnitude')
            plt.title(ttl[k])
```



Question 1.3. Explain how Welch's method works (it may be helpful to consult the help of `welch`), and the fundamental differences between this approach and the AR parametric method.

Answer 1.3. The `welch` function estimates the power spectral density (PSD) of a signal using Welch's method. According to its help, the function divides the input signal into several overlapping segments (the length is set by `nperseg`). Each segment is windowed to reduce spectral leakage and then transformed using the Fast Fourier Transform (FFT). The squared magnitudes of these FFTs are averaged to obtain a smoother and more reliable PSD estimate. In this experiment, the parameters used are:

1. `fs = 1.0` → the sampling frequency in Hz,
2. `nperseg = 100` → the number of samples per segment,
3. `nfft = 1000` → the number of FFT points, which defines the frequency resolution,
4. `scaling = 'density'` → the output is normalized to represent power per Hz.

This averaging process reduces the variance of the PSD estimate but also lowers frequency resolution.

Compared to the AR parametric method, `welch` does not assume any specific signal model. It computes the spectrum directly from the data, so it is more robust and less sensitive to parameter choices. The AR method, instead, is model-based and can provide higher resolution for short signals, but its accuracy depends heavily on the chosen model order.

In summary, welch provides a stable, data-driven PSD estimate, while the AR method gives sharper results but requires careful tuning of model parameters.

Question 1.4. In our particular problem, how do the spectra obtained with Welch's method compare with those of the parametric approach (order 15)? Given the previous RR-interval description, which approach performs most reliably?

Answer 1.4. When comparing the spectra obtained with Welch's method and the AR parametric approach (order 15), both methods show similar physiological peaks. However, the spectral shape and resolution differ noticeably. The AR(15) model provides sharper and narrower peaks, giving high frequency resolution even with a relatively short signal. It can clearly separate the low-frequency and high-frequency components of the RR-interval, but the result depends strongly on the chosen model order. If the order were slightly incorrect, the AR estimate could introduce artificial peaks or miss real ones. The Welch method, on the other hand, produces a smoother spectrum with slightly broader peaks. Because it is data-driven and does not rely on model assumptions, it is more stable and less sensitive to parameter selection. Given the physiological description of the RR-interval and the moderate data length, the AR method with order 15 captures the expected peaks more distinctly and aligns well with known heart rate variability features. However, Welch's method provides a more robust and reliable estimate overall, especially when signal noise or model uncertainty is considered. In summary, AR(15) offers better resolution, while Welch provides greater reliability. For this dataset, Welch's spectrum is the more trustworthy representation of the underlying physiological rhythms.

We now consider the signals in `/data/heart_2.dat`, which are recordings from the same individual after alcohol consumption.

```
In [8]: with open(fhr2, 'r') as f:
        txt = f.readlines()
        txt = [s[:-1].split() for s in txt]

        rr2 = np.array([float(s[0]) for s in txt])
        bp2 = np.array([float(s[1]) for s in txt])
        rs2 = np.array([float(s[2]) for s in txt])
```

Question 1.5. Repeat the processing steps done for `heart_1.dat` to obtain a parametric spectral estimation (order 15). Which of the physiological mechanisms described for the RR-interval was most disrupted by alcohol? And what other changes do you observe in the physiological signals?

```
In [9]: ***Answer 1.5.**

        # Resampling to 1 Hz and removing mean
        df = 4
        rr2 = resample(rr2, int(len(rr2) / df))
        bp2 = resample(bp2, int(len(bp2) / df))
        rs2 = resample(rs2, int(len(rs2) / df))

        for y in [rr2, bp2, rs2]:
            y -= np.mean(y)
```

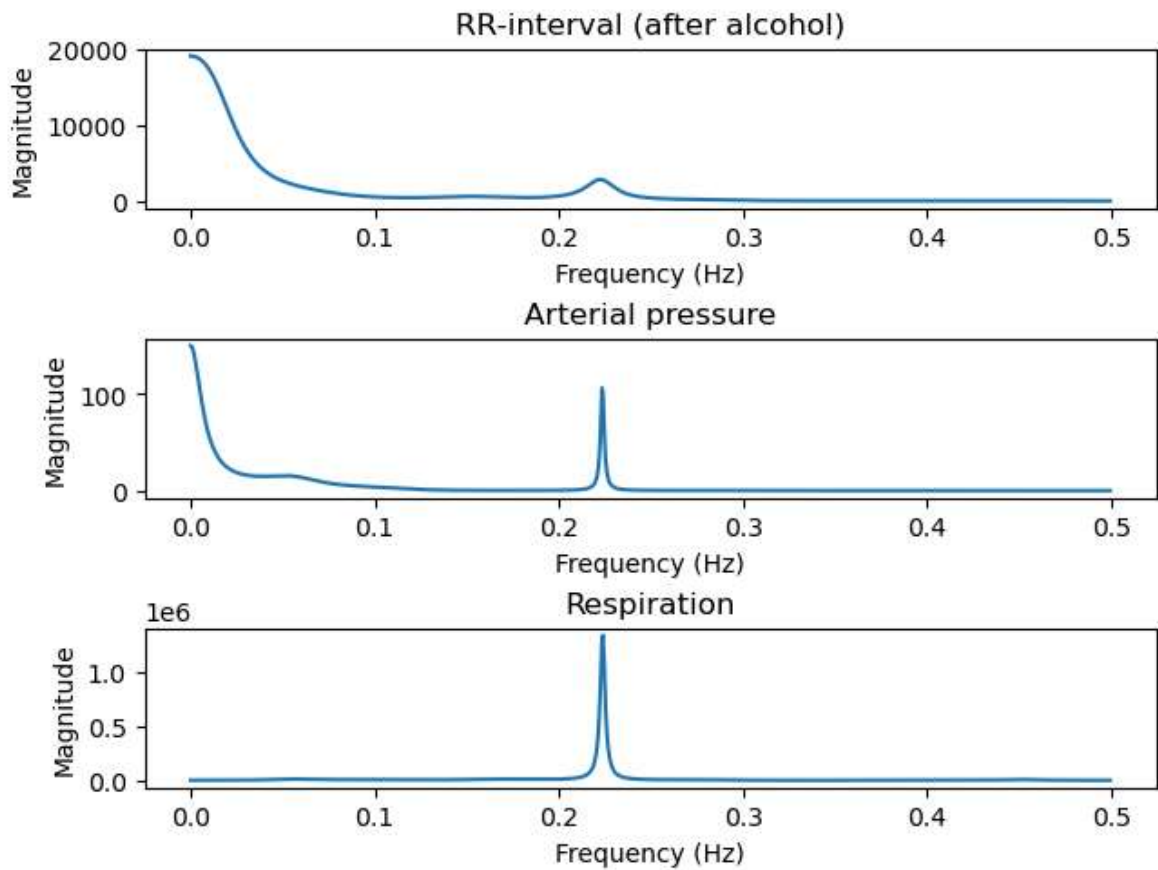
```

# Compute AR-based PSD with model order 15
signals = [rr2, bp2, rs2]
titles = ['RR-interval (after alcohol)', 'Arterial pressure', 'Respiration']

plt.figure(constrained_layout=True)

for i in range(3):
    psd, f = ar_psd(signals[i], 15, 1.0)
    plt.subplot(3, 1, i + 1)
    plt.plot(f, psd)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.title(titles[i])

```



COMMENTS

After applying the same AR(15) spectral estimation to the post-alcohol data, clear changes appear in the spectra. In the RR-interval spectrum, the low-frequency peak around 0.08 Hz—which corresponds to the baroreflex mechanism—has almost disappeared, while the high-frequency peak near 0.225 Hz, related to respiration, is still visible but reduced in amplitude. This indicates a suppression of autonomic control, especially the sympathetic–parasympathetic regulation of heart rate. The arterial pressure spectrum shows a single dominant peak near 0.22 Hz, meaning that respiration-driven oscillations remain but the slower cardiovascular regulation has weakened. The respiration spectrum itself stays almost unchanged, confirming that the breathing frequency was stable and that the disruption is primarily cardiovascular rather than respiratory. Overall, alcohol consumption has strongly reduced heart rate variability (HRV) and impaired the baroreflex mechanism, diminishing the adaptive coupling between heart rate and blood pressure.

Experiment 2: estimating the effects of fatigue on muscle signals

The files `/data/emg1.dat` and `/data/emg2.dat` contain three surface electromyography (sEMG) signals each (1024 Hz sampling frequency). These signals were recorded from the thigh muscle during a 10-min exercise on an ergocycle, respectively from an athlete (`emg1`) and from a moderately trained subject (`emg2`). In each file, the columns correspond to recordings from the *start*, *middle* and *end* of the exercise, respectively.

We start by importing the signals:

```
In [10]: with open(fem1, 'r') as f:
          txt = f.readlines()
          txt = [s[:-1].split() for s in txt]

          n = len(txt)
          em1 = np.zeros((n,3))
          for k in range(3):
              em1[:,k] = [float(s[k]) for s in txt]

          with open(fem2, 'r') as f:
              txt = f.readlines()
              txt = [s[:-1].split() for s in txt]

              n = len(txt)
              em2 = np.zeros((n,3))
              for k in range(3):
                  em2[:,k] = [float(s[k]) for s in txt]

          fs = 1024.0 # Hz
```


We first plot the sEMG segments in the time domain. We can observe four bursts in each recording, which correspond to four pedal strokes. The goal of the experiment was to assess the effects of fatigue on the signal properties of the sEMG.

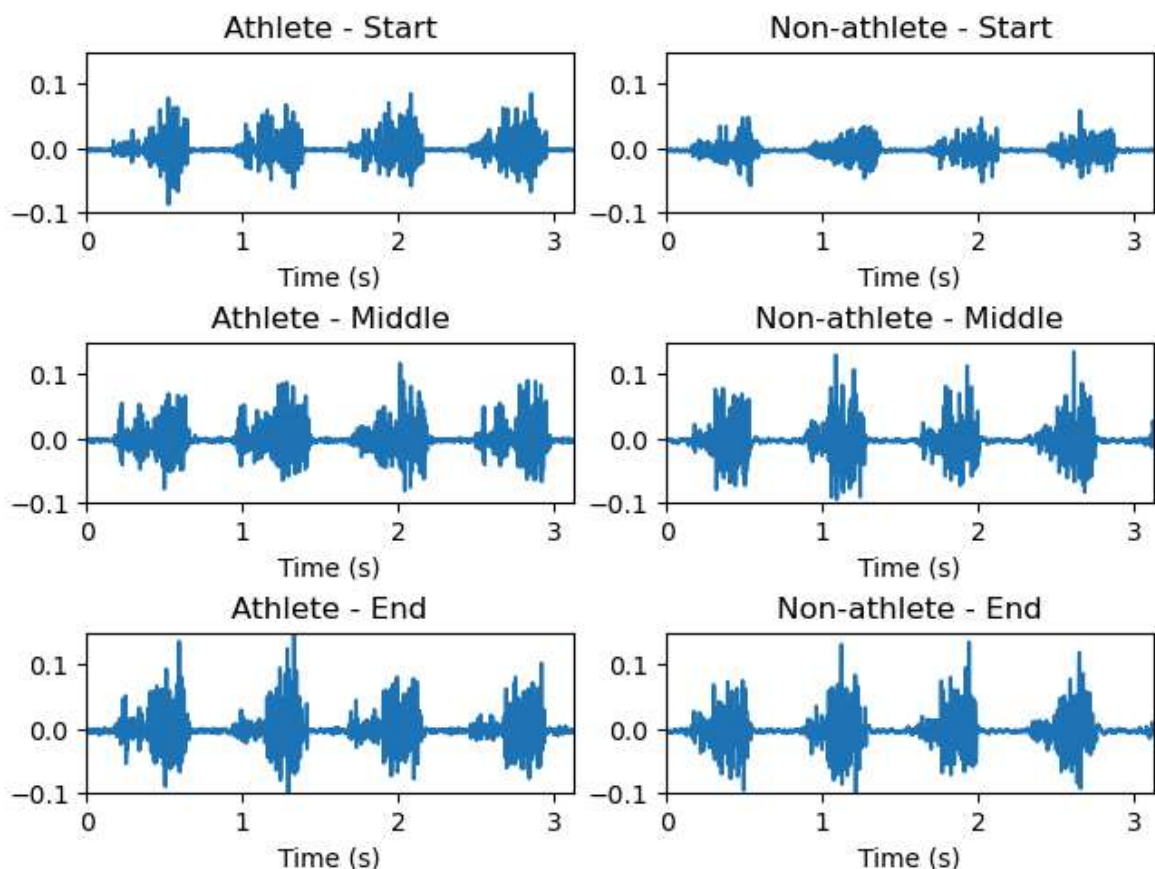
```
In [11]: tt11 = ['Athlete', 'Non-athlete']
tt12 = ['Start', 'Middle', 'End']
t = np.arange(n) / fs

plt.figure(constrained_layout=True)
for k in range(2):

    y = [em1, em2][k]

    for l in range(3):

        plt.subplot(3,2,2*l+k+1)
        plt.plot(t,y[:,l])
        plt.xlim(t[0], t[-1])
        plt.ylim(-0.10, 0.15)
        plt.xlabel('Time (s)')
        plt.title(tt11[k] + ' - ' + tt12[l])
```



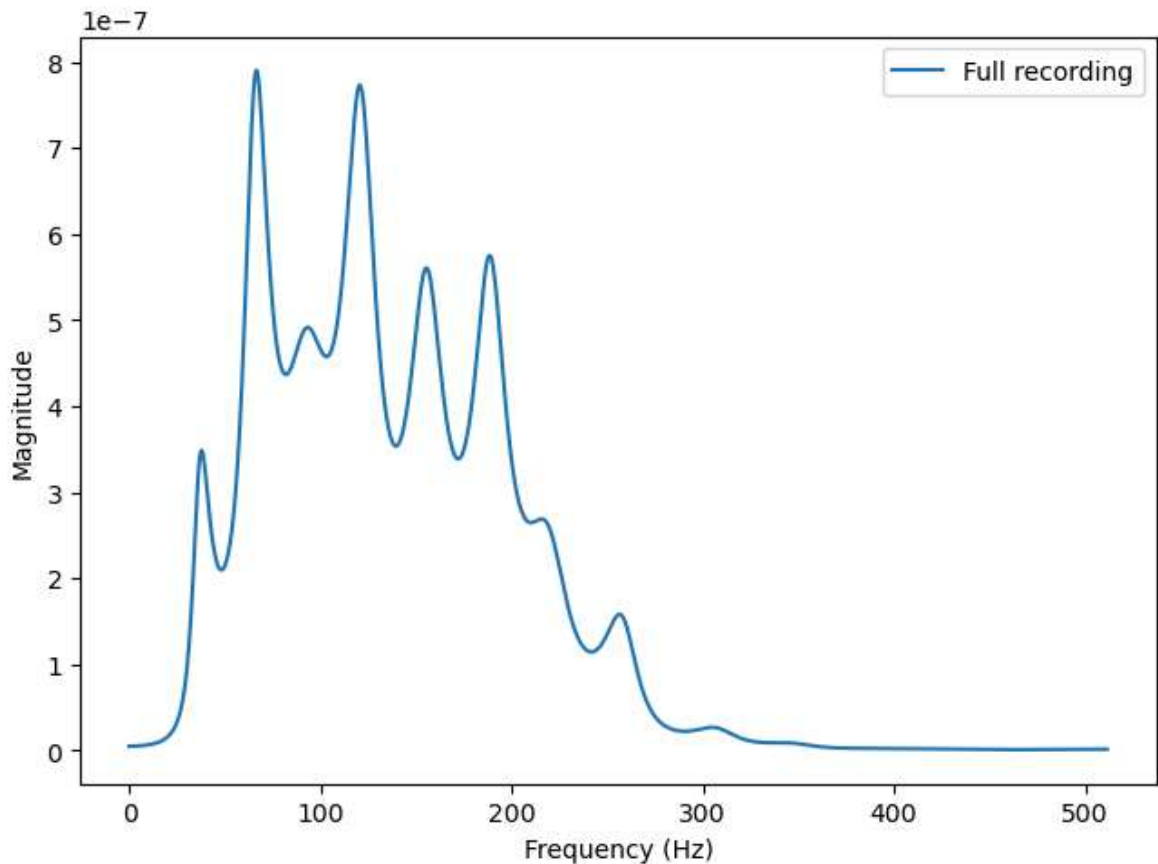
We'll now look at the spectral profile of the signals. Considering, for instance, the start sEMG recording of the athlete, we obtain the parametric power spectral density (PSD) for an AR model of order 30:

```
In [12]: p = 30
pya, fa = ar_psd(em1[:,0], p, fs)

plt.figure(constrained_layout=True)
```

```
plt.plot(fa, pya, label='Full recording')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.legend()
```

Out[12]: <matplotlib.legend.Legend at 0x2937a777a00>



Question 2.1. In principle, it would be more correct to segment each burst and analyze it separately, but it turns out that using the entire recording will yield a relatively similar spectral profile. Check that it is the case: copy the plotting code above to the space below and add (overlayed in the same panel) the PSD obtained from the first burst only, for the same signal. How do their profiles compare? Where does the difference in peak signal-to-noise come from?

Answer 2.1. Write your answer here

```
In [13]: # **Answer 2.1.**

p = 30
pya, fa = ar_psd(em1[:,0], p, fs)

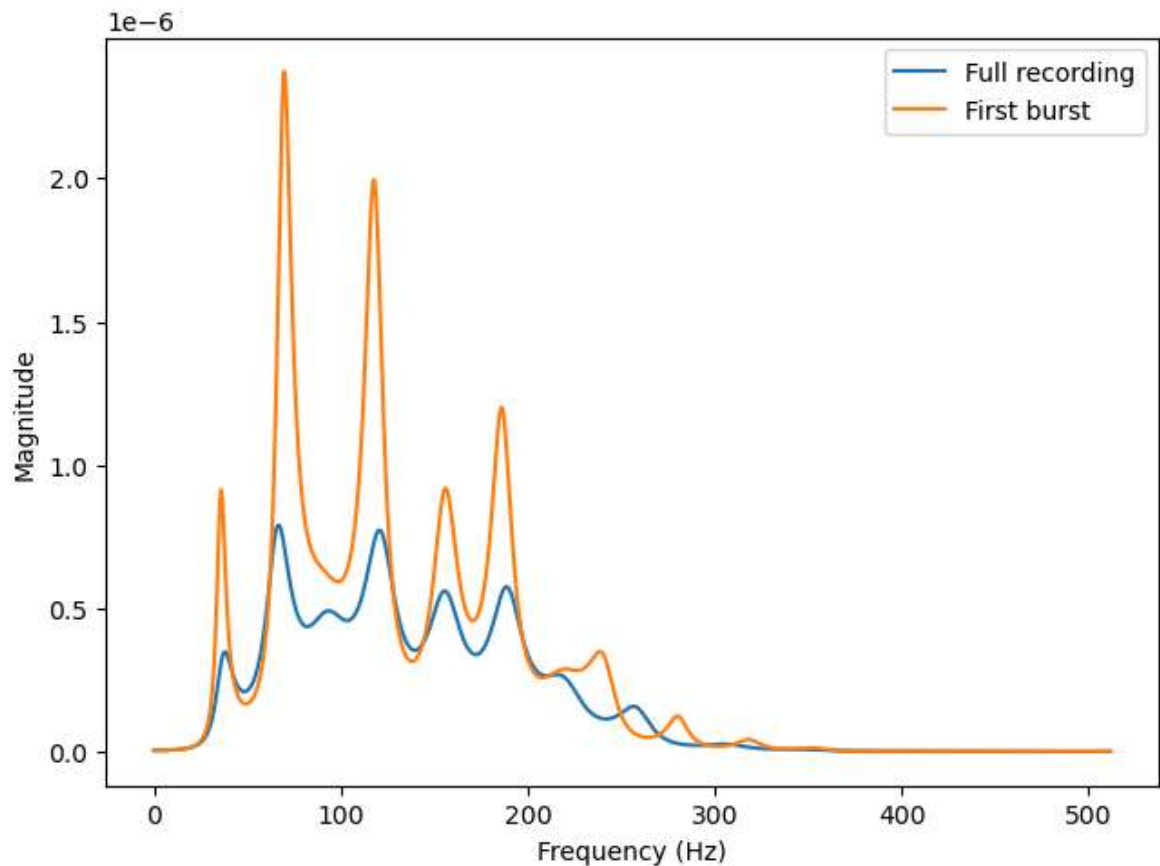
x = em1[:,0]
#based on figure 6 we can see that the first burst begin at around 148 and end a
burst = x[146:649]

pyb, fb = ar_psd(burst, p, fs)
plt.figure(constrained_layout=True)
plt.plot(fa, pya, label='Full recording')
plt.plot(fb, pyb, label='First burst')
plt.xlabel('Frequency (Hz)')
```



```
plt.ylabel('Magnitude')
plt.legend()
```

Out[13]: <matplotlib.legend.Legend at 0x2937ab297e0>



The two spectra have a very similar overall shape, with the main peaks appearing at the same frequencies and having comparable widths. However, in the full recording, the amplitude of these peaks is clearly lower. This occurs because the complete signal includes both the active contraction phases (bursts) and the silent periods between them. Since the power spectral density is computed over the entire duration, these low-activity segments reduce the average power of the main frequency components. In contrast, when only the burst segment is analyzed, the energy is concentrated in the active part of the signal, leading to sharper and higher peaks in the spectrum.

We now estimate and plot the PSD for every recorded segment of both individuals:

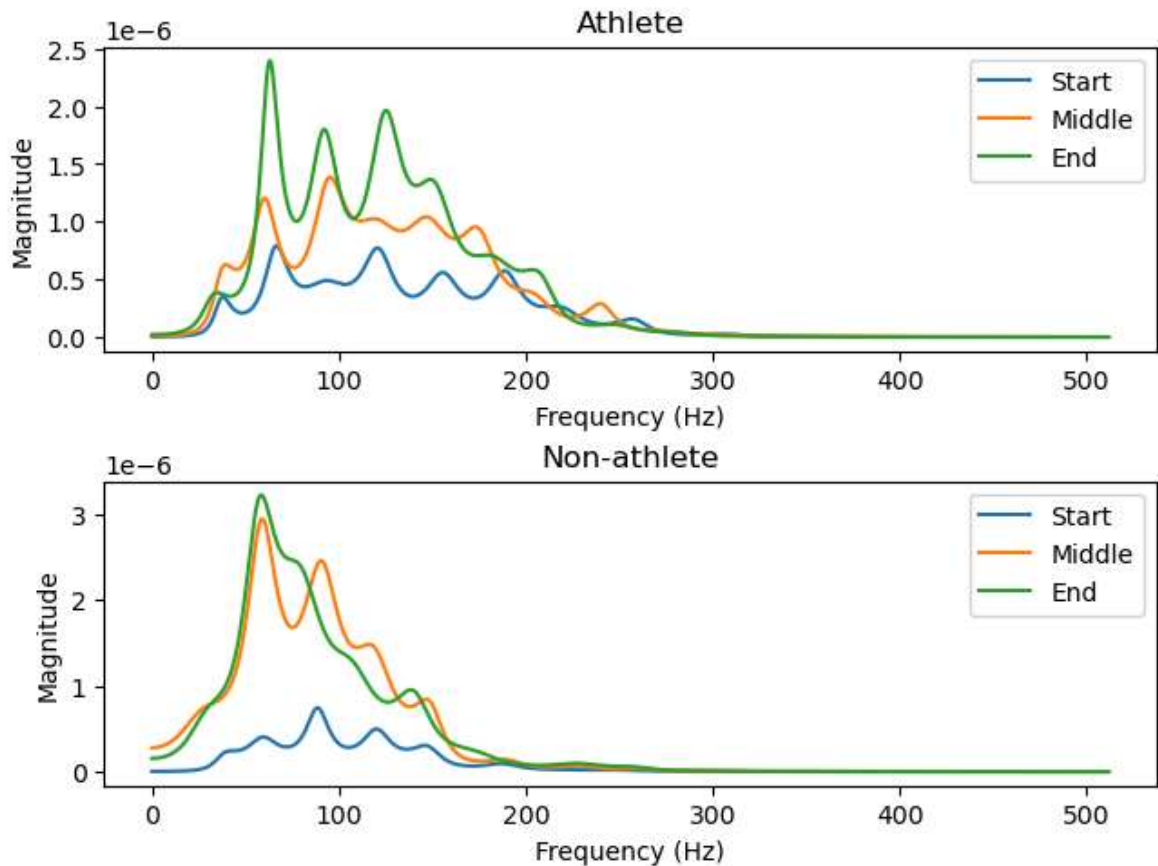
```
In [14]: plt.figure(constrained_layout=True)
for k in range(2):

    y = [em1, em2][k]
    plt.subplot(2,1,k+1)

    for l in range(3):

        pya, fa = ar_psd(y[:,l], p, fs)
        plt.plot(fa, pya, label=ttl2[l])
        plt.xlabel('Frequency (Hz)')
        plt.ylabel('Magnitude')
```

```
plt.title(tt11[k])
plt.legend()
```



Question 2.2. To complement the above PSD's, estimate the mean frequency using `sum(fa * pya / sum(pya))`. What does this formula express?

Answer 2.2. Write your answer here

```
In [15]: for k, subject in enumerate([em1, em2]):
          print(f"\n--- {tt11[k]} ---")    # Athlete / Non-athlete

          for l, phase in enumerate(tt12): # Start / Middle / End
              pya, fa = ar_psd(subject[:, l], p, fs)
              mean_freq = np.sum(fa * pya / np.sum(pya))
              print(f"{phase:<10s} -> Mean frequency = {mean_freq:.2f} Hz")
```

```
--- Athlete ---
Start      -> Mean frequency = 136.51 Hz
Middle     -> Mean frequency = 127.31 Hz
End        -> Mean frequency = 121.10 Hz
```

```
--- Non-athlete ---
Start      -> Mean frequency = 106.30 Hz
Middle     -> Mean frequency = 87.91 Hz
End        -> Mean frequency = 87.61 Hz
```

The formula gives the average frequency weighted by power, meaning frequencies with higher power contribute more. It is essentially the center of gravity of the spectrum, showing where most of the signal's energy lies.

Question 2.3. Based on the PSD plots and the mean frequencies obtained above, what marks the difference between the athlete and the moderately trained subject?

Answer 2.3. Write your answer here

The athlete's EMG signals present sharper spectral peaks and a slightly higher mean frequency that decreases gradually over time, showing better fatigue resistance. In contrast, the moderately trained subject displays broader peaks and a stronger drop in mean frequency, indicating greater fatigue and reduced muscle efficiency. The PSDs confirm this, as the non-athlete's spectrum changes abruptly between the start and middle phases, while the athlete's evolves more progressively.

Experiment 3: assessment of PPG signal structure

Photoplethysmography (PPG) is a simple, low-cost, non-invasive optical monitoring technique that can be used to detect volumetric changes in blood in the peripheral circulation, by measuring on the skin surface. It can be integrated in wearable devices (e.g. a watch) to monitor a person's heart rate throughout daily life, and detect potential issues like atrial fibrillation. The file `/data/ppg.dat` contains a PPG recording from a normal cardiac rhythm, and another from a subject in atrial fibrillation (both sampled at 21.3 Hz).

We start by importing and plotting the two signals:

```
In [16]: with open(fppg, 'r') as f:
          txt = f.readlines()
          txt = [s[:-1].split() for s in txt]

          ppg1 = np.array([float(s[0]) for s in txt])
          ppg2 = np.array([float(s[1]) for s in txt])

          for y in [ppg1, ppg2]:
              y -= np.mean(y)

          fs = 21.3 # Hz

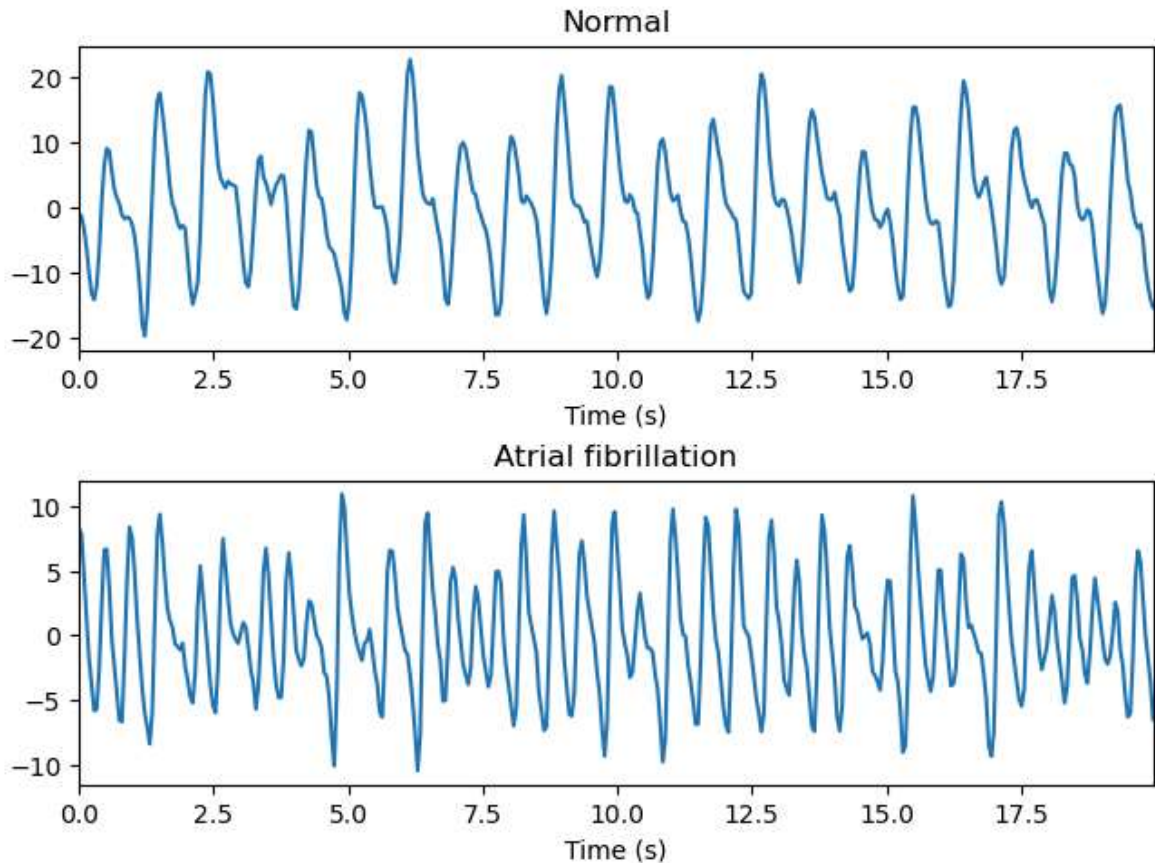
          plt.figure(constrained_layout=True)
          ttl = ['Normal', 'Atrial fibrillation']

          n = len(ppg1)
          t = np.arange(n) / fs

          for k in range(2):

              y = [ppg1, ppg2][k]

              plt.subplot(2,1,k+1)
              plt.plot(t, y)
              plt.xlim(t[0], t[-1])
              plt.xlabel('Time (s)')
              plt.title(ttl[k])
```



Each observed oscillation on the PPG typically corresponds to one pulse, generated by one cardiac cycle. By inspecting the plot above, we can already appreciate how one signal is visibly less regular than the other. AR modeling allows us to quantify the underlying differences with several complementary metrics.

We define again the function `ar_order` introduced in the previous practical session:

```
In [17]: def ar_order(x, omax, Aff=0):
        """
        AR order estimation
        x: signal
        omax: maximum possible order
        Aff: 0 no graphic display; 1 display

        Returns:
        omdl: order estimated with MDL
        """

        nx = len(x)
        s = np.zeros((omax,))
        c = np.zeros((omax,))

        for k in range(omax):

            n = k+1

            ar_model = AutoReg(x, n, trend='n')
            ar_model_fit = ar_model.fit()
            sg2 = ar_model_fit.sigma2

            s[k] = sg2
```

```

        c[k] = nx * np.log(sg2) + (n+1) * np.log(nx)

    if Aff == 1:
        plt.figure()
        plt.plot(range(1, omax+1), c, 'o-')
        plt.title('Criterion')
        plt.show()

    return np.argmin(c)+1, s, c

```

Question 3.1. Estimate the optimal AR model order for each signal, using the function `ar_order` above (you can set a maximal order of 40 for the search).

Answer 3.1. Write your answer here

The optimal autoregressive (AR) model order was estimated for the two PPG signals (normal rhythm and atrial fibrillation) using the provided `ar_order` function with a maximum order of 40 and the Minimum Description Length (MDL) criterion.

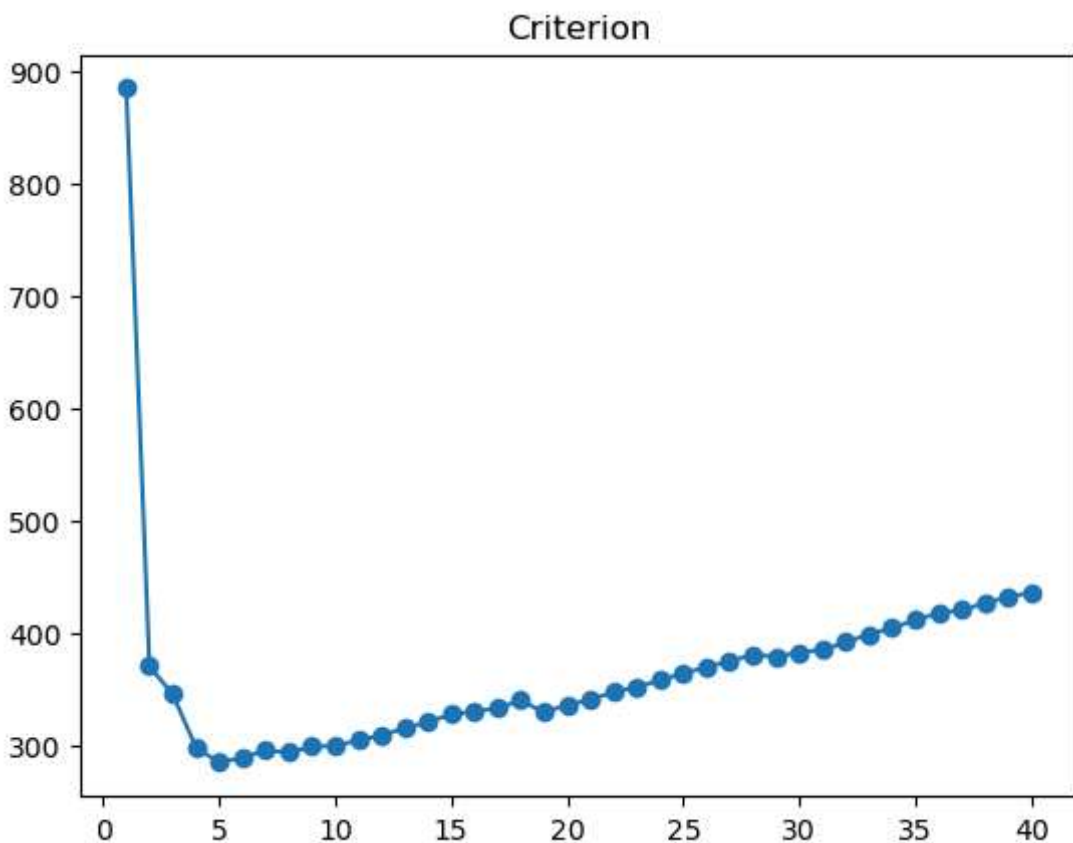
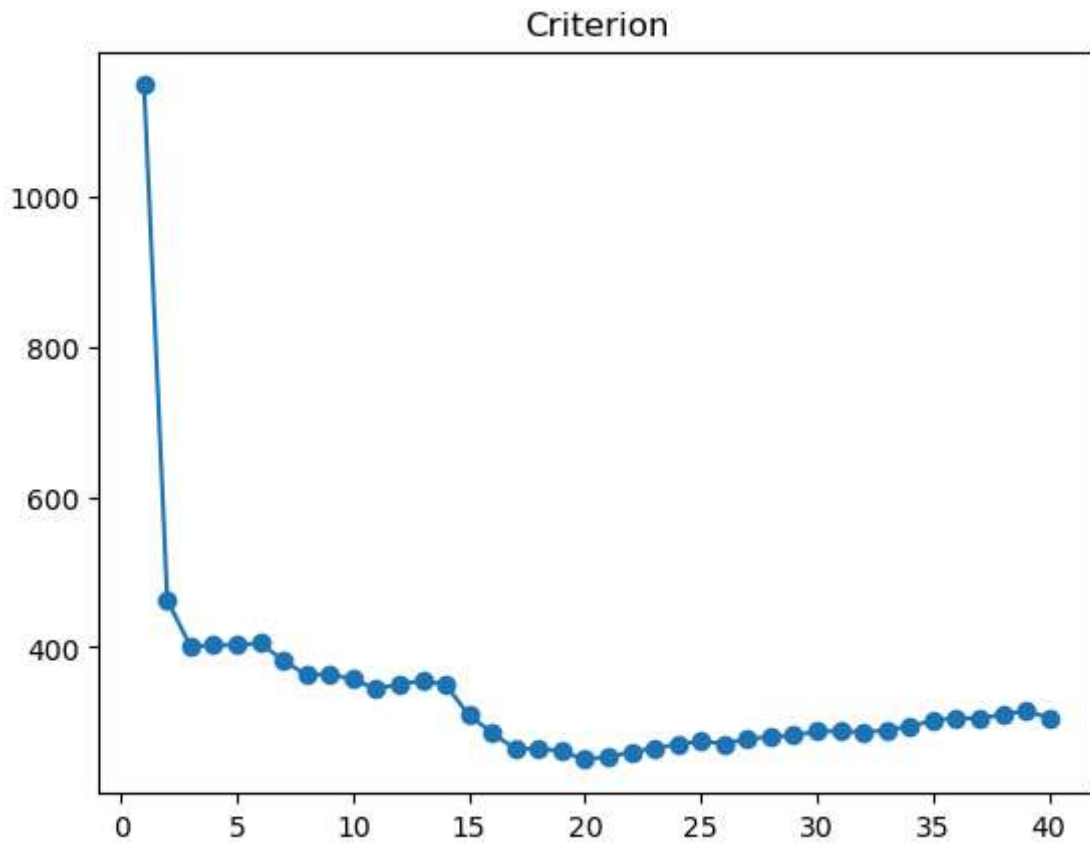
The results show an optimal order of $p = 20$ for the normal PPG and $p = 5$ for the atrial fibrillation PPG. This outcome suggests that the normal signal contains a richer and more structured spectral content, requiring a higher AR order to accurately describe its periodic components. Conversely, the atrial fibrillation signal, being more irregular and noise-like, is better represented by a low-order AR model, as additional parameters do not significantly improve the fit. Such behavior reflects the physiological difference between a regular heartbeat and the chaotic rhythm typical of atrial fibrillation.

```

In [18]: omax = 40
mdl1, s1, c1 = ar_order(ppg1, omax, Aff=1)
mdl2, s2, c2 = ar_order(ppg2, omax, Aff=1)

print(f"Normal PPG → optimal AR order (MDL): {mdl1}")
print(f"Atrial fibrillation PPG → optimal AR order (MDL): {mdl2}")

```



Normal PPG → optimal AR order (MDL): 20

Atrial fibrillation PPG → optimal AR order (MDL): 5

In []:

Question 3.2. Estimate the ratio between the excitation variance and the signal variance, for each signal (as in previous exercises, you can rely on the `yule_walker` function with `method='mle'`)

Answer 3.2. Write your answer here

The ratio between the excitation variance (σ_e^2) and the signal variance (σ_x^2) was estimated using the Yule–Walker equations with maximum likelihood, adopting the optimal AR orders obtained in Question 3.1. The following code was used:

```
In [28]: import inspect
print(inspect.getsource(yule_walker))
```



```

def yule_walker(x, order=1, method="adjusted", df=None, inv=False,
               demean=True):
    """
    Estimate AR(p) parameters from a sequence using the Yule-Walker equations.

    Adjusted or maximum-likelihood estimator (mle)

    Parameters
    -----
    x : array_like
        A 1d array.
    order : int, optional
        The order of the autoregressive process. Default is 1.
    method : str, optional
        Method can be 'adjusted' or 'mle' and this determines
        denominator in estimate of autocorrelation function (ACF) at
        lag k. If 'mle', the denominator is n=X.shape[0], if 'adjusted'
        the denominator is n-k. The default is adjusted.
    df : int, optional
        Specifies the degrees of freedom. If `df` is supplied, then it
        is assumed the X has `df` degrees of freedom rather than `n`.
        Default is None.
    inv : bool
        If inv is True the inverse of R is also returned. Default is
        False.
    demean : bool
        True, the mean is subtracted from `X` before estimation.

    Returns
    -----
    rho : ndarray
        AR(p) coefficients computed using the Yule-Walker method.
    sigma : float
        The estimate of the residual standard deviation.

    See Also
    -----
    burg : Burg's AR estimator.

    Notes
    -----
    See https://en.wikipedia.org/wiki/Autoregressive\_moving\_average\_model for
    further details.

    Examples
    -----
    >>> import statsmodels.api as sm
    >>> from statsmodels.datasets.sunspots import load
    >>> data = load()
    >>> rho, sigma = sm.regression.yule_walker(data.endog, order=4,
    ...                                     method="mle")

    >>> rho
    array([ 1.28310031, -0.45240924, -0.20770299,  0.04794365])
    >>> sigma
    16.808022730464351
    """
    # TODO: define R better, look back at notes and technical notes on YW.
    # First link here is useful
    # http://www-stat.wharton.upenn.edu/~steele/Courses/956/ResourceDetails/YuleW

```

alkerAndMore.htm

```
method = string_like(
    method, "method", options=("adjusted", "unbiased", "mle")
)
if method == "unbiased":
    warnings.warn(
        "unbiased is deprecated in favor of adjusted to reflect that the "
        "term is adjusting the sample size used in the autocovariance "
        "calculation rather than estimating an unbiased autocovariance. "
        "After release 0.13, using 'unbiased' will raise.",
        FutureWarning,
    )
    method = "adjusted"

if method not in ("adjusted", "mle"):
    raise ValueError("ACF estimation method must be 'adjusted' or 'MLE'")
x = np.array(x, dtype=np.float64)
if demean:
    if not x.flags.writeable:
        x = np.require(x, requirements="W")
    x -= x.mean()
n = df or x.shape[0]

# this handles df_resid ie., n - p
adj_needed = method == "adjusted"

if x.ndim > 1 and x.shape[1] != 1:
    raise ValueError("expecting a vector to estimate AR parameters")
r = np.zeros(order+1, np.float64)
r[0] = (x ** 2).sum() / n
for k in range(1, order+1):
    r[k] = (x[0:-k] * x[k:]).sum() / (n - k * adj_needed)
R = toeplitz(r[:-1])

try:
    rho = np.linalg.solve(R, r[1:])
except np.linalg.LinAlgError as err:
    if 'Singular matrix' in str(err):
        warnings.warn("Matrix is singular. Using pinv.", ValueWarning)
        rho = np.linalg.pinv(R) @ r[1:]
    else:
        raise

sigmasq = r[0] - (r[1:]*rho).sum()
if not np.isnan(sigmasq) and sigmasq > 0:
    sigma = np.sqrt(sigmasq)
else:
    sigma = np.nan
if inv:
    return rho, sigma, np.linalg.inv(R)
else:
    return rho, sigma
```

```
In [29]: p1 = mdl1 # normal
p2 = mdl2 # atrial fibrillation

rho1, sigma_e1 = yule_walker(ppg1, order=p1, method='mle')
rho2, sigma_e2 = yule_walker(ppg2, order=p2, method='mle')
```

```

sigma_x1 = np.var(ppg1)
sigma_x2 = np.var(ppg2)

ratio1 = sigma_e1**2 / sigma_x1 #squared due to cell above
ratio2 = sigma_e2**2 / sigma_x2 #yule walker returns std dev, need variance

print(f"Normal PPG →  $\sigma_e^2/\sigma_x^2 = \{ratio1\}$ ")
print(f"Atrial fibrillation PPG →  $\sigma_e^2/\sigma_x^2 = \{ratio2\}$ ")

```

Normal PPG → $\sigma_e^2/\sigma_x^2 = 0.027664189552700818$

Atrial fibrillation PPG → $\sigma_e^2/\sigma_x^2 = 0.10872695890695387$

The obtained ratios were 0.028 for the normal PPG and 0.0109 for the atrial fibrillation PPG. The higher value for the atrial fibrillation signal indicates that a larger portion of its variance cannot be captured by the autoregressive model, reflecting its irregular and less predictable nature. Conversely, the low ratio in the normal PPG confirms a highly regular and well-structured waveform that can be effectively modeled using AR dynamics.

Question 3.3. Estimate and plot the PSD of each signal, using the `ar_psd` function for the estimates (for consistency, use the same order of 20 for both signals)

Answer 3.3. Write your answer here

The Power Spectral Density (PSD) of the two PPG signals was estimated using the provided `ar_psd` function, which implements an autoregressive (AR) model-based spectral estimation via the Yule–Walker equations. To ensure comparability between signals, both PSDs were computed using an AR model of order $p = 20$ and a sampling frequency of 21.3 Hz.

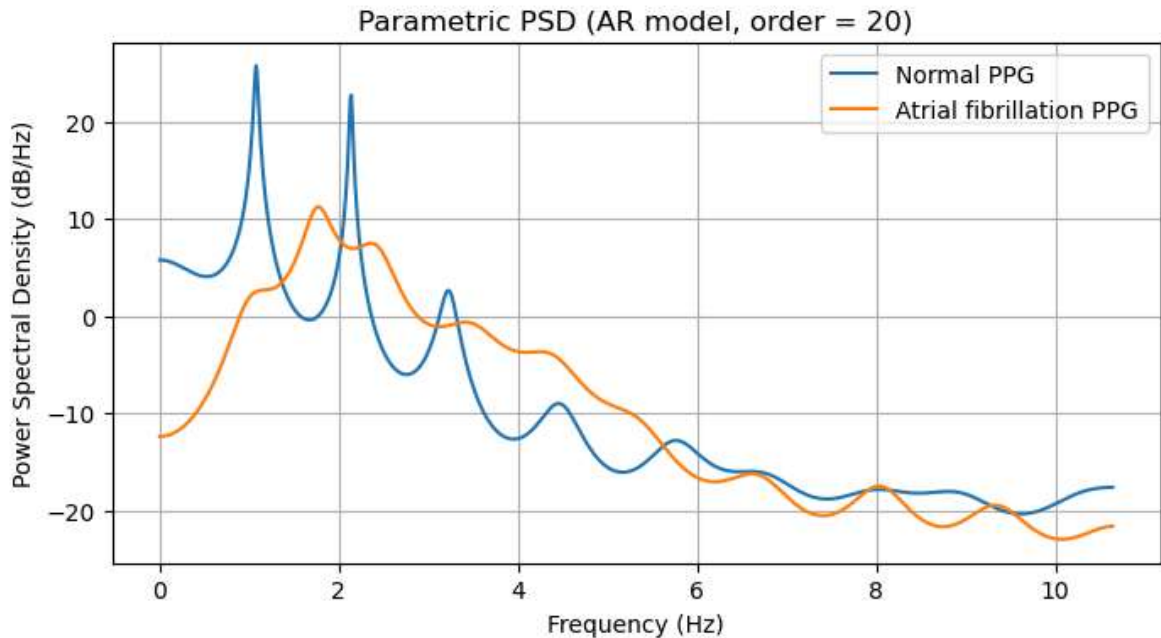
```

In [20]: fs = 21.3    # Sampling frequency [Hz]
         p = 20      # Fixed AR order for both signals

psd1, f1 = ar_psd(ppg1, p, fs)
psd2, f2 = ar_psd(ppg2, p, fs)

# Plot
plt.figure(figsize=(8,4))
plt.plot(f1, 10*np.log10(psd1), label='Normal PPG')
plt.plot(f2, 10*np.log10(psd2), label='Atrial fibrillation PPG')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power Spectral Density (dB/Hz)')
plt.title('Parametric PSD (AR model, order = 20)')
plt.legend()
plt.grid(True)
plt.show()

```



The resulting PSDs highlight the spectral differences between the two cardiac conditions. The normal PPG exhibits a dominant and narrow spectral peak corresponding to the fundamental heart rate frequency and its harmonics, consistent with a stable and quasi-periodic cardiac rhythm. The atrial fibrillation PPG, instead, displays a broader and less defined spectrum, reflecting irregular pulse intervals and reduced spectral coherence.

In []:

Question 3.4. Estimate the entropy of the PSD of each signal, using:

$$p = py / \text{sum}(py)$$

$$\text{entropy} = \text{sum}(-p * \log(p))$$

where py is the vector of PSD amplitudes.

Answer 3.4. Write your answer here

```
In [21]: # Compute spectral entropy according to the given formula
p1 = psd1 / np.sum(psd1)
p2 = psd2 / np.sum(psd2)

entropy1 = np.sum(-p1 * np.log(p1))
entropy2 = np.sum(-p2 * np.log(p2))

print(f"Normal PPG → Spectral entropy = {entropy1:.3f}")
print(f"Atrial fibrillation PPG → Spectral entropy = {entropy2:.3f}")
```

Normal PPG → Spectral entropy = 4.007

Atrial fibrillation PPG → Spectral entropy = 5.504

The atrial fibrillation PPG shows a higher entropy, indicating that its spectral energy is distributed over a broader frequency range, consistent with the irregular and less coherent temporal pattern of the signal. The normal PPG, in contrast, exhibits a lower entropy, reflecting that most of its spectral energy is concentrated around the dominant heart rate frequency.

Question 3.5. What do these measures (model order, variance ratio, PSD, entropy) tell us about the two signals? Interpret what you have obtained in terms of signal structure/organization.

Answer 3.5. Write your answer here

The four measures (AR model order, excitation-to-signal variance ratio, parametric PSD, and spectral entropy) consistently reveal the fundamental differences between the two PPG signals. The normal PPG is characterized by:

- a higher AR model order ($p = 20$), reflecting a richer but well-organized temporal structure;
- a low excitation-to-signal variance ratio ($\sigma_e^2/\sigma_x^2 \approx 0.018$), indicating that most of the signal variability is predictable from its past values;
- a narrow, distinct spectral peak in the PSD, corresponding to the stable heart rate frequency and its harmonics;
- a low spectral entropy, confirming that the signal energy is concentrated in a few dominant frequency components.

In contrast, the atrial fibrillation PPG shows:

- a lower AR model order ($p = 5$), suggesting a less structured and more irregular temporal pattern;
- a higher excitation-to-signal variance ratio ($\sigma_e^2/\sigma_x^2 \approx 0.072$), indicating that a larger fraction of its variance is unpredictable;
- a broader PSD without a clear dominant peak, consistent with the irregular rhythm of atrial fibrillation;
- a higher spectral entropy, reflecting a more uniform and disorganized spectral energy distribution.

Overall, these results demonstrate that the normal PPG exhibits strong temporal and spectral regularity, while the atrial fibrillation PPG is characterized by spectral dispersion, reduced predictability, and increased randomness — all indicative of the loss of rhythmic coordination in cardiac activity.

Experiment 4: Pisarenko harmonic estimation of brain signals

The file `/data/eeg_stim.dat` contains three recordings of scalp electroencephalography (EEG; left frontal electrode, 4-s duration, 512 Hz sampling frequency) from a Parkinson's patient. The patient was also implanted with a deep brain stimulation (DBS) electrode. The first recording corresponds to a baseline (rest) state, the second was obtained during 1-Hz stimulation, and the third during 100-Hz stimulation.

We start by importing and plotting the three signals:

```
In [22]: with open(feeg, 'r') as f:
          txt = f.readlines()
```

```

txt = [s[:-1].split() for s in txt]

n = len(txt)
eeg = np.zeros((n,3))
for k in range(3):
    eeg[:,k] = [float(s[k]) for s in txt]

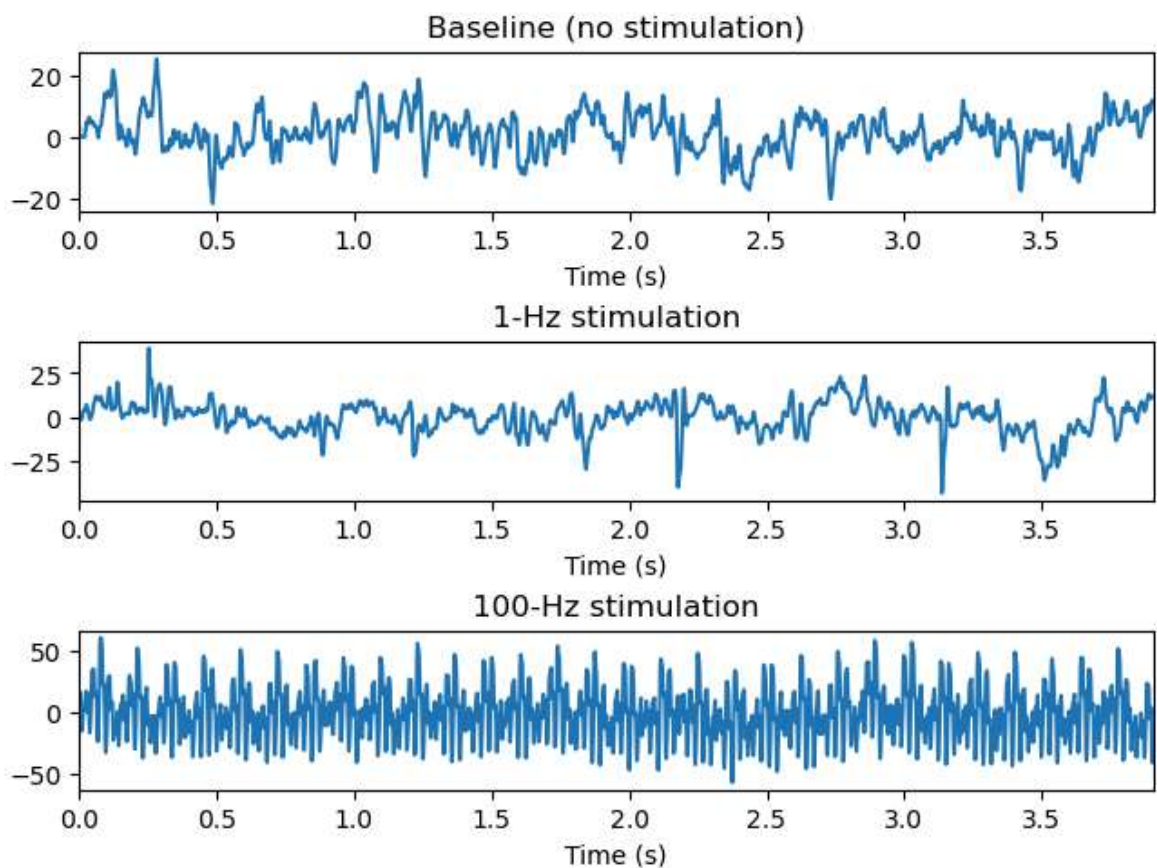
fs = 512.0 # Hz

plt.figure(constrained_layout=True)
ttl = ['Baseline (no stimulation)', '1-Hz stimulation', '100-Hz stimulation']

t = np.arange(n) / fs

for k in range(3):
    plt.subplot(3,1,k+1)
    plt.plot(t, eeg[:,k])
    plt.xlim(t[0], t[-1])
    plt.xlabel('Time (s)')
    plt.title(ttl[k])

```



Question 4.1. Visually, does any of the stimulation strategies appear to have an effect on the EEG? Which one(s)?

Answer 4.1. Yes, visually, one of the stimulation strategies appears to have a very clear effect on the EEG:
the 100-Hz stimulation.

- Baseline vs. 1-Hz stimulation: These two plots look very similar. Both show irregular fluctuations with amplitudes (the y-axis values) that mostly stay within the

approximate -25 to +25 range. There isn't a clear, systematic difference between them.

- Baseline vs. 100-Hz stimulation: Here, the difference is clearly visible. The 100-Hz signal shows:

Greater Amplitude: The activity oscillates roughly between -50 and +50, which is more than double the amplitude of the baseline signal.

Higher Frequency: The signal oscillates much more rapidly (there are many more "wiggles" packed into the same amount of time) compared to the slower, broader waves in the baseline signal.

As in previous examples, we can apply modeling techniques to quantify the physiological effects under study. Because the DBS is applied in a periodic regime with a specific frequency, it is reasonable to expect a strong sinusoidal presence on the EEG recordings, for any cases where the stimulation does produce a measurable effect. This motivates applying the Pisarenko harmonic retrieval approach, which is implemented below:

```
In [23]: def pisarenko(x, p, fs, Aff=0):
        """
        Pisarenko method for harmonic retrieval

        x: signal
        p: number of sinusoids
        fs: sampling frequency
        Aff: 0 no graphic display; 1 display

        f: vector of sinusoid frequencies
        pp: vector of sinusoid amplitudes
        sg2: noise variance

        """

        n = len(x)

        rx = correlate(x, x, mode='full', method='auto') / n

        rxx = np.zeros((2*p+1, 2*p+1))
        for k1 in range(2*p+1):
            for k2 in range(2*p+1):
                rxx[k1, k2] = rx[n-1 + abs(k1 - k2)]

        u, v = np.linalg.eig(rxx)

        sg2 = np.min(u)
        vm = v[:, np.argmaxmin(u)] # get eigenvector from smallest eigenvalue

        r = np.roots(vm) # find roots of the polynomial
        f = np.angle(r) / (2*np.pi) # get angle and convert to Hz/sample
        f = f[f > 0]

        # Amplitude estimation
        aa = np.zeros((p, len(f)))
        for k in range(p):
            aa[k, :] = np.cos(2*np.pi * (k+1) * f)
```



```

r = rx[n:n+p]

pp = np.linalg.lstsq(aa, r, rcond=None)[0]
f *= fs

if Aff:

    plt.figure(constrained_layout=True)

    plt.subplot(2,1,1)
    plt.plot(np.arange(1,n+1) / fs, x)
    plt.xlabel('Time (s)')
    plt.title('Signal')

    plt.subplot(2,1,2)
    plt.plot([0, fs/2], [sg2, sg2], linewidth=2)
    for k in range(p):
        plt.plot([f[k], f[k]], [sg2, sg2+pp[k]], linewidth=2)
    plt.axis([0, fs/2, 0, 1.2 * (np.max(pp) + sg2)])
    plt.xlabel('Frequency (Hz)')
    plt.title('Pisarenko spectrum')

return f, pp, sg2

```

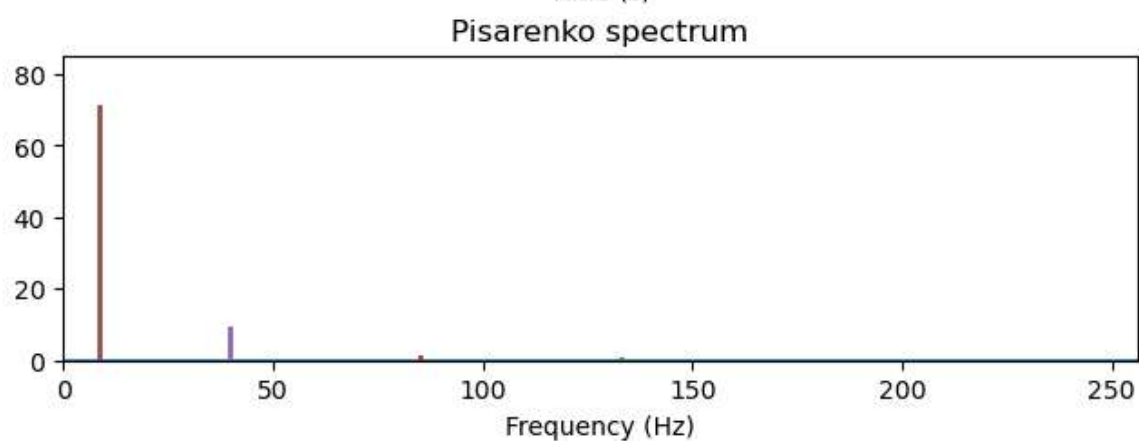
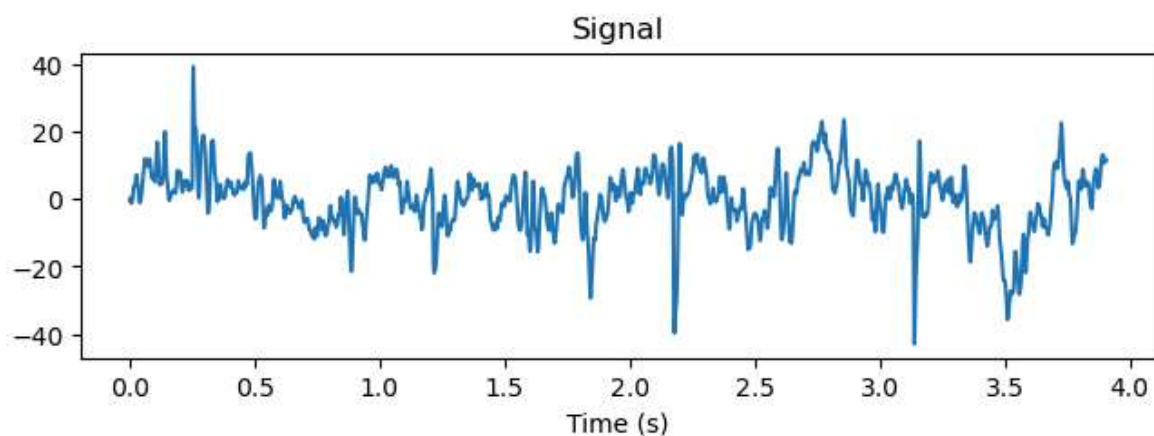
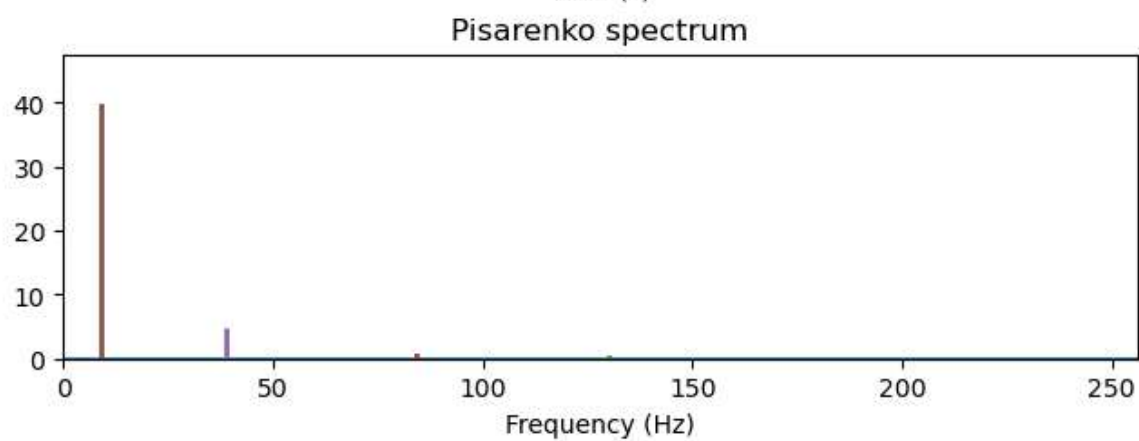
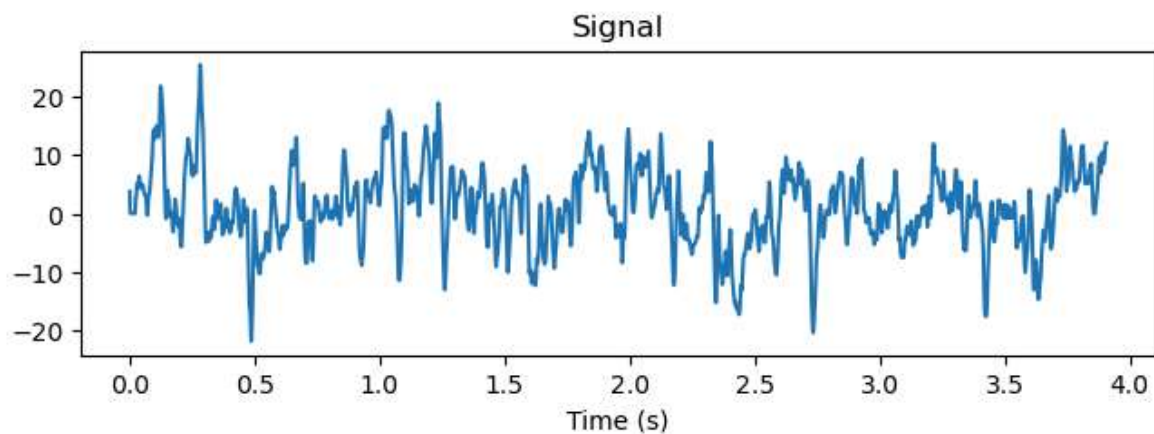
Question 4.2. Using the above function, apply the Pisarenko method with 5 sinusoids to each of the three recordings, and report the estimated parameters (frequencies, amplitudes, excitation variance).

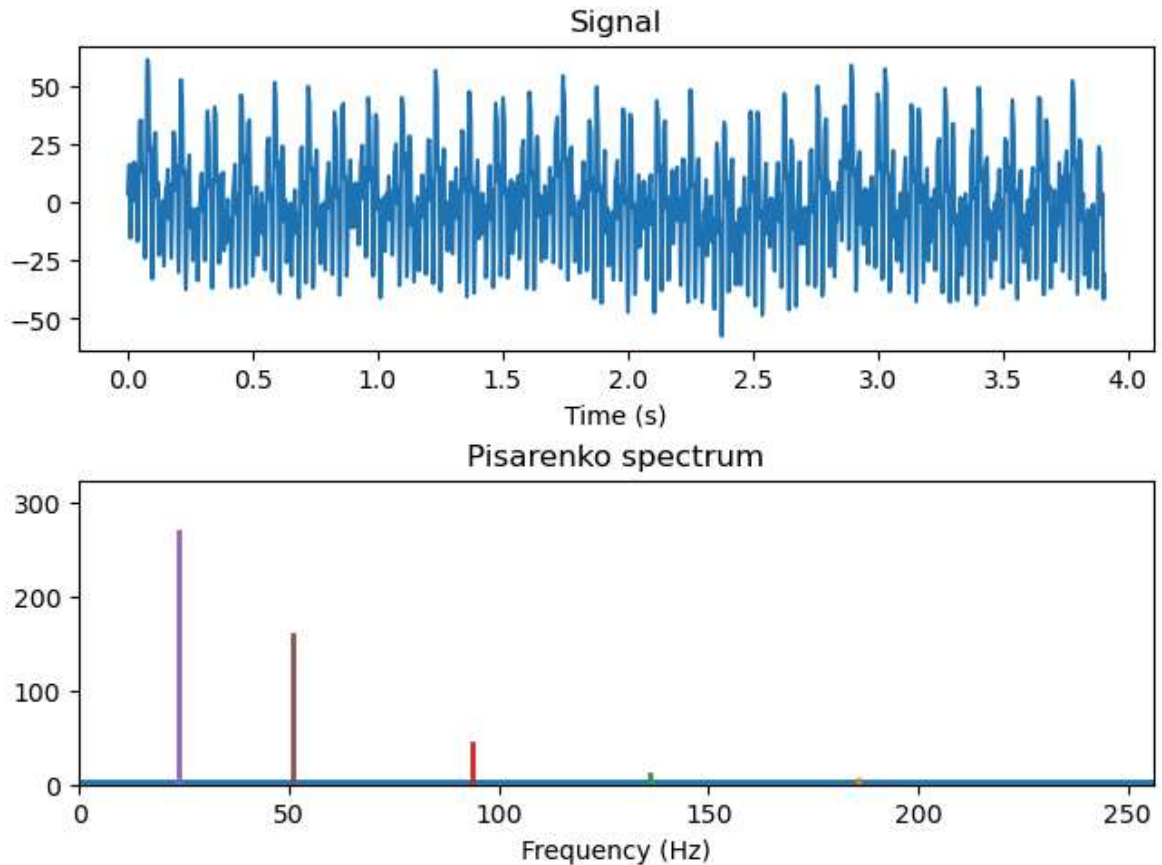
Answer 4.2. Write your answer here

```

In [24]: f_baseline, pp_baseline, sg2_baseline = pisarenko(x=eeg[:, 0], p=5, fs=fs, Aff=1)
f_stim1, pp_stim1, sg2_stim1 = pisarenko(x=eeg[:, 1], p=5, fs=fs, Aff=1)
f_stim100, pp_stim100, sg2_stim100 = pisarenko(x=eeg[:, 2], p=5, fs=fs, Aff=1)

```





Question 4.3. Interpret the estimation results -- which stimulation frequency appears to effectively influence brain activity?

Answer 4.3. Based on the Pisarenko spectra, the 100-Hz stimulation appears to be the only one that effectively influences brain activity.

- Baseline vs. 1-Hz Stimulation

When we compare the spectrum for the baseline (third image) with the spectrum for the 1-Hz stimulation (first image), they are almost identical. Baseline: Shows a prominent spike at a very low frequency (under 10 Hz) and a much smaller one around 40 Hz. These represent the natural, underlying rhythms in this patient's brain. 1-Hz Stimulation: Shows the exact same pattern. There is no new, significant spike at or near 1 Hz. The low-frequency spike looks identical to the baseline.

This suggests the 1-Hz stimulation had no measurable effect on the EEG's frequency content.

- Baseline vs. 100-Hz Stimulation In sharp contrast, the spectrum for the 100-Hz stimulation (second image) is dramatically different from the baseline. Several new, very strong spectral peaks (the vertical lines) appear that were completely absent in the baseline plot. We can clearly see powerful new components at approximately 50 Hz, another one very close to 100 Hz, and even others at higher frequencies (like ~140 Hz and ~190 Hz). The original low-frequency activity seen in the baseline is still present, but it is now tiny compared to the massive new peaks introduced by the stimulation.

Conclusion

The appearance of these powerful new frequency components, which are directly related to the stimulation frequency (100 Hz and its sub-harmonic at 50 Hz), is clear evidence that the 100-Hz stimulation is effectively influencing and fundamentally altering the brain's electrical activity. The 1-Hz stimulation shows no such effect.

Have a good session, and don't hesitate to ask questions!

In []: